# LAPORAN PRAKTIKUM

# PEMROGRAMAN BERORIENTASI OBJEK

**MODUL 4&5**

**Contractor**

**Method &**

**Encapsulation**

**Disusun Oleh:**

**Ahmad Zaidan 105224044**

**PROGRAM STUDI ILMU KOMPUTER**

**FAKULTAS SAINS DAN KOMPUTER**

**UNIVERSITAS PERTAMINA**

**2026**

## I. Pendahuluan

Pada praktikum Modul 4 & 5 ini, mahasiswa mempelajari konsep lanjutan Pemrograman Berorientasi Objek yaitu **Encapsulation**, **Constructor**, dan **Method Overloading**. Encapsulation digunakan untuk melindungi data dengan membatasi akses langsung ke atribut melalui modifier private dan menyediakan metode getter serta setter dengan validasi bisnis yang ketat. Constructor dipelajari sebagai metode khusus untuk inisialisasi objek, sementara Method Overloading memungkinkan pembuatan beberapa metode dengan nama yang sama namun parameter berbeda guna meningkatkan fleksibilitas program.

## II. Variabel

| No | Nama Variabel | Tipe Data | Fungsi |
|---|---|---|---|
| 1 | nomorKamar | String | Gunanya untuk menyimpan identitas nomor kamar hotel |
| 2 | tipeKamar | String | Menyimpan kategori kamar |
| 3 | kapasitasMaksimal | Int | Untuk batas maksimum tamu yang menempati kamar |
| 4 | hargaPerMalam | Double | Untuk nominal biaya sewa kamar per satu malam |
| 5 | IsTersedia | Boolean | Untuk status ketersediaan kamar, true menandakan kamar kosong kalau false menandakan kamar terisi |
| 6 | jumlahMalam | Int | Untuk parameter jumlah durasi menginap untuk perhitungan tagihan |
| 7 | kodeVoucher | String | Untuk parameter guna mamvalidasi penggunaan diskon promo |
| 8 | jumlahTamu | Int | Untuk parameter guna validasi kapasitas saat pemesanan kamar |

## III. Dokumentasi dan Pembahasan Code

THT > src > KamarHotel.java

```java
public class KamarHotel {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;

    // Constructor Pertama: Pendaftaran Kilat
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

    // Constructor Kedua: Pendaftaran Lengkap
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam);
        this.isTersedia = true;
    }

    // Getter untuk semua atribut
    public String getNomorKamar(){
        return nomorKamar;
    }
    public String getTipeKamar(){
        return tipeKamar;
    }
    public int getKapasitasMaksimal(){
        return kapasitasMaksimal;
    }
    public double getHargaPerMalam(){
        return hargaPerMalam;
    }
    public boolean isTersedia(){
        return isTersedia;
    }

    // Setter dengan validasi ketat
    public void setTipeKamar(String tipe) {
        if (tipe.equalsIgnoreCase(anotherString: "Reguler") || tipe.equalsIgnoreCase(anotherString: "Premium") || tipe.equalsIgnoreCase(anotherString: "Suite"))
            this.tipeKamar = tipe;
        } else {
            System.out.println("Peringatan: Tipe '" + tipe + "' tidak valid. Otomatis diatur ke 'Reguler'.");
            this.tipeKamar = "Reguler";
        }
```

src > KamarHotel.java

```java
public class KamarHotel {

    public void setHargaPerMalam(double harga) {
        if (harga < 50000) {
            System.out.println(x: "Peringatan: Harga di bawah batas minimal. Otomatis diatur ke 50000.");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = harga;
        }
    }

    // Method Overloading: Pemesanan Kamar
    public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan.");
        } else {
            System.out.println("Peringatan: Kamar " + nomorKamar + " sudah terisi!");
        }
    }

    public void pesanKamar(int jumlahTamu) {
        if (!isTersedia) {
            System.out.println(x: "Peringatan: Kamar sudah terisi!");
        } else if (jumlahTamu > kapasitasMaksimal) { // Validasi kapasitas
            System.out.println("Pesanan Ditolak: Jumlah tamu (" + jumlahTamu + ") melebihi kapasitas (" + kapasitasMaksimal + ").");
        } else {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan untuk " + jumlahTamu + " tamu.");
        }
    }

    public void batalPesan() {
        this.isTersedia = true;
        System.out.println("Pesanan kamar " + nomorKamar + " telah dibatalkan.");
    }

    public double hitungTotalBayar(int jumlahMalam) {
        return hargaPerMalam * jumlahMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equals(anObject: "PROMO") && jumlahMalam >= 3) {
            System.out.println(x: "assikk dapat diskon 20%.");
            return total * 0.8;
        } else {
            System.out.println(x: "yahh sayang bgt kode salah atau durasi menginap kurang dari 3 malam.");
            return total;
        }
    }

    // menampilkan struk kamar
    public void tampilkanInfoKamar() {
        System.out.println(x: "STRUK INFORMASI KAMAR");
        System.out.println("Nomor Kamar   : " + nomorKamar);
        System.out.println("Tipe Kamar    : " + tipeKamar);
        System.out.println("Kapasitas Max : " + kapasitasMaksimal + " orang");
        System.out.println("Harga/Malam   : Rp" + hargaPerMalam);
        System.out.println("Status        : " + (isTersedia ? "Tersedia" : "Terisi"));
    }
}
```

Main.java  KamarHotel.java M

THT > src > Main.java > Main > main(String[])

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {
        // 1. Uji Kamar Pertama
        System.out.println(x: "Uji Kamar 1:");
        KamarHotel kamar1 = new KamarHotel(nomorKamar: "A1", tipeKamar: "Presidential", kapasitasMaksimal: 4, -10000);
        kamar1.tampilkanInfoKamar();

        // 2. Uji Kamar Kedua
        System.out.println(x: " Uji Kamar 2:");
        KamarHotel kamar2 = new KamarHotel(nomorKamar: "A2", tipeKamar: "Suite", kapasitasMaksimal: 2);
        kamar2.setHargaPerMalam(harga: 100000); // Set harga manual karena awalnya 0

        kamar2.pesanKamar(jumlahTamu: 4); // Coba pesan 4 orang (max 2)
        kamar2.pesanKamar(jumlahTamu: 2); // Terus pesan kamar itu pakai jumlah tamu misalnya 2 orang
        kamar2.pesanKamar(); // Coba lakuin pemesanan lagi tapi ga pakai parameter pesanKamar() pada kamar yang sama u

        // Uji Penagihan
        System.out.println(x: "Uji Penagihan:");
        System.out.println(x: "Tagihan Kamar 1 (2 malam, PROMO):");
        double tagihanKamar1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        System.out.println("Total Bayar: Rp" + tagihanKamar1);

        // Hitung tagihan kamar kedua untuk 4 malam dengan voucher "PROMO"
        System.out.println(x: "Tagihan Kamar 2 (4 malam, PROMO):");
        double tagihanKamar2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
        System.out.println("Total Bayar: Rp" + tagihanKamar2);

        // cetak struk akhir
        System.out.println(x: "\n=== STRUK INFORMASI AKHIR ===");
        kamar1.tampilkanInfoKamar();
        kamar2.tampilkanInfoKamar();
    }
}
```

**Pembahasan Kode kamarHotel.java**

```java
public class KamarHotel {
    // Encapsulation: Semua atribut private
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Bagian ini digunakan untuk mendefinisikan identitas yang dimiliki oleh setiap objek kamar. Disini menggunakan modifier private untuk menerapkan encapsulation, gunanya untuk melindungi data agar tidak di akses atau diubah dari luas kelas.

```java
    // Constructor Pertama: Pendaftaran Kilat
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

```

Digunakan untuk menginisialisasi objek kamar saat pertama kali dibuat dengan data minimal untuk tiap kamar (nomor, tipe, dan kapasitas). Fungsi utamanya untuk mempermudah pendaftaran cepat di mana sistem secara otomatis mengatur harga awal ke angka 0 dan status ketersediaan langsung menjadi true (tersedia).

```java
    // Constructor Kedua: Pendaftaran Lengkap
    public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal, double hargaPerMalam) {
        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam);
        this.isTersedia = true;
    }
```

Constructor ini melayani pembuatan objek dengan data yang lebih lengkap, termasuk nominal harga per malam. Sama dengan constructor yang pertama, status ketersediaan juga diatur otomatis menjadi true agar kamar siap digunakan dalam sistem.

```java
    // Getter untuk semua atribut
    public String getNomorKamar(){
        return nomorKamar;
    }
    public String getTipeKamar(){
        return tipeKamar;
    }
    public int getKapasitasMaksimal(){
        return kapasitasMaksimal;
    }
    public double getHargaPerMalam(){
        return hargaPerMalam;
    }
    public boolean isTersedia(){
        return isTersedia;
    }

```

Karena atribut tersebut bersifat private, metode **Getter** berfungsi sebagai satu-satunya solusi untuk mengambil atau melihat nilai atribut dari luar kelas tanpa memberikan izin untuk mengubahnya.

```java
    // Setter dengan validasi ketat
    public void setTipeKamar(String tipe) {
        if (tipe.equalsIgnoreCase(anotherString: "Reguler") || tipe.equalsIgnoreCase(anotherString: "Premium") || tipe.equalsIgnoreCase(anotherString: "Suite")) {
            this.tipeKamar = tipe;
        } else {
            System.out.println("Peringatan: Tipe '" + tipe + "' tidak valid. Otomatis diatur ke 'Reguler'.");
            this.tipeKamar = "Reguler";
        }
    }

```

Berfungsi untuk mengubah tipe kamar dengan validasi bisnis yang ketat. Sistem hanya mengizinkan input "Reguler", "Premium", atau "Suite" tanpa mempedulikan huruf besar/kecil. Jika input tidak sesuai, sistem akan mencetak peringatan dan memaksa tipe menjadi "Reguler" untuk menjaga konsistensi data.

```java
    public void setHargaPerMalam(double harga) {
        if (harga < 50000) {
            System.out.println(x: "Peringatan: Harga di bawah batas minimal. Otomatis diatur ke 50000.");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = harga;
        }
    }

```

Berfungsi untuk mengatur harga sewa kamar dengan batasan minimum. Jika harga yang dimasukkan di bawah 50rb , sistem akan menolak dan otomatis mengubahnya menjadi 50rb untuk mencegah kerugian dan kesalahan input harga yang minus atau tidak logis.

```java
    // Method Overloading: Pemesanan Kamar
    public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan.");
        } else {
            System.out.println("Peringatan: Kamar " + nomorKamar + " sudah terisi!");
        }
    }

```

Merupakan bagian dari **Method Overloading** untuk pemesanan cepat. Fungsinya adalah langsung mengubah status kamar menjadi tidak tersedia (false) jika kamar kosong, atau memberikan peringatan jika kamar tersebut ternyata sudah dipesan.

```java
    public void pesanKamar(int jumlahTamu) {
        if (!isTersedia) {
            System.out.println(x: "Peringatan: Kamar sudah terisi!");
        } else if (jumlahTamu > kapasitasMaksimal) { // Validasi kapasitas
            System.out.println("Pesanan Ditolak: Jumlah tamu (" + jumlahTamu + ") melebihi kapasitas (" + kapasitasMaksimal + ").")
        } else {
            isTersedia = false;
            System.out.println("Kamar " + nomorKamar + " berhasil dipesan untuk " + jumlahTamu + " tamu.");
        }
    }
```

Versi kedua dari metode pemesanan yang lebih spesifik. Selain mengecek ketersediaan, metode ini memvalidasi apakah jumlah tamu melebihi kapasitas maksimal kamar atau tidak. Jika tamu terlalu banyak, pesanan otomatis ditolak untuk menjaga operasional hotel.

```java
    public void batalPesan() {
        this.isTersedia = true;
        System.out.println("Pesanan kamar " + nomorKamar + " telah dibatalkan.");
    }

    public double hitungTotalBayar(int jumlahMalam) {
        return hargaPerMalam * jumlahMalam;
    }
```

Berfungsi untuk membatalkan transaksi yang sudah ada dan mengembalikan status kamar kembali menjadi tersedia (true) agar bisa dipesan oleh tamu lain. Lalu di bagian hitungTotalBayar Digunakan untuk menghitung total tagihan standar berdasarkan lama menginap. Metode ini mengembalikan hasil perkalian antara harga per malam dengan jumlah malam yang diinput.

```java
    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
        double total = hitungTotalBayar(jumlahMalam);
        if (kodeVoucher.equals(anObject: "PROMO") && jumlahMalam >= 3) {
            System.out.println(x: "assikk dapat diskon 20%.");
            return total * 0.8;
        } else {
            System.out.println(x: "yahh sayang bgt kode salah atau durasi menginap kurang dari 3 malam.");
            return total;
        }
    }

```

Merupakan Method Overloading untuk sistem penagihan dengan fitur promo. Logikanya adalah memberikan diskon 20% jika tamu menggunakan kode voucher "PROMO" dan menginap minimal 3 malam. Jika syarat tidak terpenuhi, sistem akan memberikan pesan penolakan dan mengenakan harga normal.

```java
    // menampilkan struk kamar
    public void tampilkanInfoKamar() {
        System.out.println(x: "STRUK INFORMASI KAMAR");
        System.out.println("Nomor Kamar   : " + nomorKamar);
        System.out.println("Tipe Kamar    : " + tipeKamar);
        System.out.println("Kapasitas Max : " + kapasitasMaksimal + " orang");
        System.out.println("Harga/Malam   : Rp" + hargaPerMalam);
        System.out.println("Status        : " + (isTersedia ? "Tersedia" : "Terisi"));
    }
}
```

Untuk mencetak seluruh informasi atau profil kamar secara rapi ke layar (seperti struk). Metode ini menampilkan kondisi akhir dari semua atribut objek, memberikan laporan yang jelas bagi pengguna sistem

**Pembahasan kode Main.java**

```java
    // 1. Uji Kamar Pertama
    System.out.println(x: "Uji Kamar 1:");
    KamarHotel kamar1 = new KamarHotel(nomorKamar: "A1", tipeKamar: "Presidential", kapasitasMaksimal: 4, -10000);
    kamar1.tampilkanInfoKamar();
```

Bagian ini berfungsi untuk menguji ketahanan sistem terhadap input data yang tidak valid. Misalkan program coba menginput tipe kamar "Presidential" (yang tidak terdaftar) dan harga negatif "-10000". Tujuannya untuk membuktikan kalau metode setter di kelas KamarHotel berhasil melakukan validasi otomatis dengan memaksa tipe kamar menjadi "Reguler" dan harga menjadi "50000".

```java
        // 2. Uji Kamar Kedua
        System.out.println(x: " Uji Kamar 2:");
        KamarHotel kamar2 = new KamarHotel(nomorKamar: "A2", tipeKamar: "Suite", kapasitasMaksimal: 2);
        kamar2.setHargaPerMalam(harga: 100000); // Set harga manual karena awalnya 0

        kamar2.pesanKamar(jumlahTamu: 4); // Coba pesan 4 orang (max 2)
        kamar2.pesanKamar(jumlahTamu: 2); // Terus pesan kamar itu pakai jumlah tamu misalnya 2 orang
        kamar2.pesanKamar(); // Coba lakuin pemesanan lagi tapi ga pakai parameter pesanKamar() pada kamar yang sama untuk

```

Bagian ini digunakan untuk menguji logika pemesanan kamar melalui tiga scenario. Pertama, validasi kapasitas dengan mencoba memesan untuk 4 orang di kamar berkapasitas 2 orang untuk memastikan sistem memblokir pesanan karena *overcapacity*. Kedua, pemesanan berhasil dengan melakukan pemesanan ulang dengan jumlah tamu yang sesuai (2 orang) hingga status kamar berubah menjadi terisi. Terakhir, pencegahan duplikasi dengan cara mencoba memesan kembali kamar yang sudah terisi untuk membuktikan sistem memberikan peringatan bahwa kamar tidak tersedia.

```java
        // Uji Penagihan
        System.out.println(x: "Uji Penagihan:");
        System.out.println(x: "Tagihan Kamar 1 (2 malam, PROMO):");
        double tagihanKamar1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        System.out.println("Total Bayar: Rp" + tagihanKamar1);
```

Bagian ini bertujuan untuk menguji logika diskon pada metode hitungTotalBayar. Meskipun menggunakan kode voucher "PROMO", sistem harus tetap memberikan harga normal karena durasi menginap hanya 2 malam (kurang dari syarat minimal 3 malam).

```java
        // Hitung tagihan kamar kedua untuk 4 malam dengan voucher "PROMO"
        System.out.println(x: "Tagihan Kamar 2 (4 malam, PROMO):");
        double tagihanKamar2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
        System.out.println("Total Bayar: Rp" + tagihanKamar2);
```

Bagian ini memperlihatkan keberhasilan sistem dalam menerapkan promo. Karena durasi menginap adalah 4 malam dan menggunakan kode "PROMO", sistem akan otomatis memotong harga sebesar 20% sesuai dengan aturan bisnis yang sudah ditetapkan.

```java
        // cetak struk akhir
        System.out.println(x: "\n=== STRUK INFORMASI AKHIR ===");
        kamar1.tampilkanInfoKamar();
        kamar2.tampilkanInfoKamar();
    }
}
```

Bagian Ini memberikan laporan akhir yang transparan mengenai status ketersediaan dan rincian data setiap kamar di dalam sistem.

## IV. Kesimpulan

Program manajemen reservasi hotel yang telah dirancang ini merupakan wujud nyata dari penerapan prinsip-prinsip utama Pemrograman Berorientasi Objek (PBO) untuk membangun sistem yang tangguh dan minim kesalahan operasional. Melalui implementasi Encapsulation, integritas data pada atribut krusial seperti harga dan tipe kamar dapat terjaga dengan sangat ketat karena akses langsung dari luar kelas telah dibatasi menggunakan modifier private. Penggunaan metode setter yang dilengkapi dengan logika validasi bisnis memastikan bahwa setiap data yang masuk ke dalam sistem selalu berada dalam rentang yang logis, di mana sistem secara otomatis mampu mengoreksi input yang tidak valid seperti harga di bawah batas minimal atau tipe kamar yang tidak sesuai standar kembali ke nilai default yang telah ditentukan. Hal ini membuktikan bahwa arsitektur kode yang baik dapat menjadi pertahanan pertama dalam mencegah *bug* administratif maupun kerugian finansial bagi manajemen hotel.

Selain itu, fleksibilitas sistem ini sangat didukung oleh penerapan Constructor dan Mthod Overloading yang efisien. Adanya dua jenis *constructor* memungkinkan proses pendaftaran objek dilakukan secara adaptif, baik melalui jalur "pendaftaran kilat" dengan data minimal maupun "pendaftaran lengkap" yang lebih detail, tanpa mengurangi validitas status awal kamar. Di sisi lain, *method overloading* pada fungsi pemesanan dan penagihan memberikan kemampuan bagi satu metode untuk menangani berbagai skenario bisnis yang berbeda, seperti memvalidasi kapasitas tamu secara otomatis guna mencegah *overcapacity* serta menghitung potongan harga secara presisi berdasarkan penggunaan kode voucher "PROMO" dan durasi menginap. Secara keseluruhan, praktikum ini menunjukkan bahwa penggabungan validasi ketat pada *setter*, inisialisasi objek yang fleksibel, dan *overloading* metode menciptakan sistem informasi yang tidak hanya fungsional, tetapi juga reliabel dan profesional dalam menyajikan profil serta struk informasi kamar secara transparan

## IV. Daftar Pustaka

Tim Asisten Laboratorium Komputer. (2026). *Modul Praktikum Pemrograman Berorientasi Objek - Modul 4 & 5*. Program Studi Ilmu Komputer, Universitas Pertamina.